Verifiable Credentialsを基盤とした自己主権型アクセス権限委譲方式：REST APIへの適用

梅田 匠[1,a)]　掛井 将平[1,b)]　白石 善明[2,c)]　齋藤 彰一[1,d)]

概要：Webサービスの利用を通して，パーソナルデータは異なるサービス事業者に分散して蓄積されている．データ主体が自らの裁量でデータ利用を他者へ許可できれば，データ主体の同意の下でサービスを横断したデータの活用ができる．本研究では，REST APIのアクセス権限をデータ主体自身が第三者へ委譲できる，自己主権型アクセス権限委譲方式を提案する．パーソナルデータはデータ主体に帰属する一方で，それを用いた機能は事業者に帰属する．委譲の判断は両者にまたがるため，委譲はサービス事業者と協働して行えることが望ましい．提案方式では，事業者が発行するVerifiable Credential（VC）が権限委譲の資格を表し，データ主体が発行するVCが権限委譲を表す．両VCが表す権限はURLとHTTPメソッドの組で細粒度に規定され，包含判定により検証される．提案方式を標準仕様OID4VC上に構成し，概念実証により全フェーズの処理が数十ミリ秒で完了することを示した．また不正な委譲を想定した攻撃シナリオを実行し，いずれも検証点で拒否されること，および設計した検証が不可欠であることを確認した．

キーワード：アクセス権限委譲，Verifiable Credentials，自己主権型アイデンティティ，アクセス制御，REST API

A Self-Sovereign Access Permission Delegation Scheme Based on Verifiable Credentials: Application to REST APIs

Takumi Umeda[1,a)]　Shohei Kakei[1,b)]　Yoshiaki Shiraishi[2,c)]　Shoichi Saito[1,d)]

Abstract: Personal data accumulates across service providers. Enabling data subjects to authorize others to use their data would allow cross-service utilization with their consent. This paper proposes a self-sovereign access permission delegation scheme in which a data subject delegates REST API access permissions to a third party. Personal data belongs to the data subject, while functionality built on it belongs to the provider; delegation decisions span both and should be made cooperatively. The provider issues a Verifiable Credential (VC) representing eligibility to delegate, and the data subject issues a VC representing the delegation. Permissions in both VCs are fine-grained pairs of a URL and an HTTP method, verified by containment checking. The scheme is built on the OID4VC standard; a proof-of-concept shows every phase completes within tens of milliseconds. Attack scenarios of illegitimate delegation were all rejected at the verification points, and each designed verification proved indispensable.

Keywords: Access Permission Delegation, Verifiable Credentials, Self-Sovereign Identity, Access Control, REST API

1. はじめに

Webサービスの利用を通して，個人のパーソナルデータは異なるサービスに分散して蓄積されている．データ駆動型社会に向けてこれを分野横断で利活用する要請が高まっ

[1] 名古屋工業大学
[2] 神戸大学
[a)] t.umeda.323@stn.nitech.ac.jp
[b)] kakei.shohei@nitech.ac.jp
[c)] zenmei@port.kobe-u.ac.jp
[d)] shoichi@nitech.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

ており，国内外で制度・基盤の整備が進んでいる [1], [2]．その中でも，GDPR のデータポータビリティ権 [3] や，利用者主権のアイデンティティである Self-Sovereign Identity（SSI）[4] のように，データ主体主導の制御を求める流れが強まっている．

パーソナルデータはデータ主体に帰属する一方で，データの管理およびそれを用いた機能はサービス事業者に帰属する．そのため，データ主体とサービス事業者が協調してデータの利用を制御できることが望ましい．Web システムにおけるデータ連携には，Web API が広く利用されており，そのアクセス制御に OAuth2.0[5] が標準化されている．OAuth2.0 のもとでは，データ主体が自身のデータへのアクセスを第三者に認可できるが，その相手は認可サーバーが事前に許可した者に限られる．サービス事業者が事前に把握していない第三者に対しても，データ主体が自らアクセス権限を委譲でき，かつ委譲したアクセス権限とその委譲元の正当性を特定の認可サーバーに依存せず検証できれば，分散した個人のリソースの協調的な活用が期待できる．

本稿では，先行研究で提案した Verifiable Credentials（VCs）を基盤とする自己主権型アクセス権限委譲方式を REST API へ適用する．提案方式では，これを資格発行者・権限委譲者・権限被委譲者・検証者からなるモデルとして定式化し，URL パターンと HTTP メソッドの組による細粒度な権限表現と，その包含判定に基づく委譲を実現する．サービス事業者はデータ主体に対して，アクセス権限を委譲する資格を証明する VC を発行し，データ主体はこの VC を根拠として，第三者へのアクセス権限の委譲を証明する VC を発行する．本研究では，データ主体が発行する VC を Verifiable Grant（VG）と呼ぶ．VG で委譲される権限は，その根拠となる VC がデータ主体に認めた権限の範囲に束縛され，特定の第三者機関に依存せずにデータ主体がその資格を有することを検証できる．

2. 本研究のアプローチ

2.1 先行研究：Verifiable Grant（VG）

著者らは，検証可能な資格情報である Verifiable Credentials（VCs）を基盤としてサービスとデータ主体が協働するアクセス権限委譲手法 [6] を提案した．この提案では，アクセス権限をデータの識別子と CRUD 操作の組として抽象的に表現し，その委譲関係を VC をベースとした Verifiable Grant（VG）として定義した．VCs を基盤とすることで，主体が権限を直接委譲でき， 検証者は各主体の識別子と公開鍵が登録された分散台帳（Verifiable Data Registry, VDR）を参照して，委譲された権限を検証できる．

2.2 先行研究を REST API に適用するための要件

本研究では，先行研究を資格発行者・権限委譲者・権限被委譲者・検証者からなるモデルとして定式化する．資格発行者がパーソナルデータを管理し，権限委譲者にそのデータのアクセス権限を委譲する資格を発行する．権限委譲者はその権限を権限被委譲者へ委譲し，権限被委譲者は検証者に対して権限を行使する．つまり，権限委譲には，(1) 資格発行者から権限委譲者への委譲資格の発行，(2) 権限委譲者から権限被委譲者への権限委譲，(3) 権限被委譲者から検証者に対する権限行使，の 3 段の権限連鎖が生じる．

本提案では，このモデルを REST API へ適用する．REST API は，リソースを URL で識別し，それに対する操作を HTTP メソッドで指定する Web API の設計様式である．URL は，サブツリーやパスパラメーターを通じて複数の具体 URL を指すパターンとなるため，連鎖の各段の権限は同一とは限らず，具体 URL の集合の間に包含関係が生じる．これに対し，先行研究の抽象的な権限表現では，URL の階層構造やパスパラメーターの違いを区別できなかった．また，先行研究では権限の一致を確かめるだけで委譲関係を検証できたが，REST API では段をまたぐこの包含関係を確かめなければならない．加えて先行研究は，各主体の識別子と公開鍵の VDR への登録を前提とするため，委譲先が事実上その基盤の参加者に限定されていた．SSI の観点からは，委譲先が共通基盤の参加者に限定されないことが求められる [4]．以上から，以下の 3 つの要件を立てる．

要件 1：細粒度な権限表現　権限をリソースの階層構造とパスパラメーターを区別した URL と，操作を表す HTTP メソッドの組として表現すること．

要件 2：権限範囲の遡及検証　各段の権限が前段の権限に包含され，起点となる資格が資格発行者の認めた範囲に収まることを確かめられること．

要件 3：委譲先の非限定性　特定の共通基盤への参加を前提とせず，ドメインをまたいで委譲が成立すること．

要件 2 に対しては，URL を正規化したうえでの区画単位のパターン照合と，許可された HTTP メソッドの集合に対する包含判定により，権限行使 $\sqsubseteq$ 権限委譲 $\sqsubseteq$ 資格発行が成り立つことを検証する．要件 3 に対しては，資格発行者と検証者が同一の主体であることを利用する．パーソナルデータを管理する主体が委譲資格を発行し，同じ主体が権限行使を受け付けるため，検証者は連鎖の起点となる資格発行者の鍵を自らのものとして保持する．権限委譲者の鍵は委譲資格を示す VC に，権限被委譲者の鍵は VG に紐付くため，検証者は自身の鍵を起点に鍵の連鎖をたどり，各主体の鍵の正当性を確かめられる．よって識別子や鍵を VDR へ登録して検証のたびに問い合わせる必要がなく，委譲先が特定の基盤の参加者に限定されない．

© 2026 Information Processing Society of Japan

3. 提案

3.1 システム構成

3.1.1 資格情報の表現

提案方式で扱う証明とその流れを**図 1** に示す．提案方式では，権限に関する資格情報とその提示を 4 つの証明に分けて扱う．VC は資格発行者が権限委譲者へ発行する委譲資格の証明，VG は権限委譲者が権限被委譲者へ発行する委譲の証明である．VPc は VC の提示であり，VPg は VG の提示と，その根拠となる VPc の一部からなる．

提案方式の VC ／ VG は SD-JWT VC を用いて表現する．SD-JWT VC とは，要素を個別に開示および秘匿できる SD-JWT[7] を用いて，資格情報の選択的開示を可能にした VC 表現である [8]．`cnf` プロパティで保持者の鍵を紐づけでき，提示時に Key Binding JWT（KB-JWT）によって保持者証明を行う．KB-JWT には提示内容のハッシュ `sd_hash` も含まれる．また，発行者識別子を表す `iss` から発行者メタデータを解決して検証鍵を取得できるため，独自の鍵配布を必要としない．さらに，Token Status List（TSL）[9] を用いて VC の有効性を確認できる．以降，資格情報の基本検証とは，発行者署名の検証，KB-JWT による保持者証明の検証，`exp` とキャッシュ済 Status List による有効性の確認の 3 項目を指す．

3.1.2 エンティティ

提案方式は以下のエンティティで構成される．

対象サービス（資格発行者，検証者） データ主体にサービスを提供する中で，パーソナルデータを保管している事業者．資格発行者を担いデータ主体に VC を発行する VC 発行サーバーと，検証者を担いデータ利用サービスによるアクセスを受け付けるリソースサーバーを持つ．

データ主体（権限委譲者） 対象サービスの保管するデータの主体であり，VC 発行サーバーから発行された VC の管理と，データ利用サービスへの VG 発行を行うデータ主体ウォレットを持つ．

データ利用サービス（権限被委譲者） データ主体のパーソナルデータを利用するサービスを提供する事業者であり，データ主体から発行された VG を管理するデータ利用サービスウォレットを持ち，データ主体から委譲された権限を使って対象サービスのパーソナルデータにアクセスする．

3.1.3 アクセス権限の表現と包含

権限表現について，`authorization_details` によって細粒度なアクセス権限を構造的に表現する Rich Authorization Requests（RAR）を用いる [10]．RAR はフィールドの意味を `type` ごとに定めることを認めており，提案方式では `api_access` 型と定義する．また，1 つの権限を 1 つの URL パターン（`locations`）と HTTP メソッドの集合（`actions`）で表現する．`authorization_details` は権限の配列であり，SD-JWT の配列要素に対する選択的開示を用いることで，権限単位で委譲・行使を制御する．

表 1 権限の包含の例
Table 1 Example of Permission Containment.

段	URL パターン	メソッド
（ドメイン空間）	`api.health.example/`	—
資格発行（VC）	`.../devices/{id}/**`	`{GET, POST}`
委譲（VG）	`.../devices/{id}/heart_rate`	`{GET}`
行使（要求）	`.../devices/d1/heart_rate`	`{GET}`

URL パターンは，パスの各区画をリテラル，任意の 1 区画に一致するパスパラメーター`{param}`，その位置以下のすべてに一致する末尾のサブツリー`**`のいずれかで表す．`{param}`の名称は照合に用いないため，名称の異なるパターンでも同一の集合を表す．ワイルドカードは明示し，暗黙の階層包含は行わない．`actions` には HTTP メソッドをそのまま列挙し，列挙されていないメソッドは拒否する．

URL パターン P が表す具体 URL の集合を $U(P)$ とし，パターン P_A とメソッド集合 M_A からなる権限 A について $R(A) = U(P_A) \times M_A$ とすると，上位権限 B に対する包含は $A \sqsubseteq B \iff R(A) \sqsubseteq R(B)$ と定義される．比較の相手が複数の権限からなる場合は，そのいずれか 1 つに包含されることを求める．この包含は，(a) 具体リクエストとパターンの判定（行使時），(b) パターンどうしの判定（委譲時）の 2 つに用いる．包含の例を**表 1** に示す．

なお判定はパスと HTTP メソッドで行い，クエリ文字列は用いない．また判定は区画単位で行い，その前に URL の正規化を適用する．先行研究における一致判定は $R(A) = R(B)$ を求めるものであり，包含判定はその一般化にあたる．

3.2 提案方式の流れ

提案方式は，2 章の権限の連鎖の各段に対応した，委譲資格取得・権限委譲・権限行使の三つのフェーズからなる．提案方式は，VC の標準仕様群である OpenID for Verifiable Credentials（OID4VC）のうち，VC 発行を司る OpenID for Verifiable Credential Issuance（OID4VCI）[11] と，VP 提示を司る OpenID for Verifiable Presentation（OID4VP）[12] で構成される．

3.2.1 委譲資格取得フェーズ

委譲資格取得フェーズでは対象サービスがデータ主体にアクセス権限の委譲資格を証明する VC を発行する．このフェーズは，OID4VCI の Pre-Authorized Code フロー上に構成され，Issuer を VC 発行サーバー，Holder をデータ主体とする．

フェーズの詳細を**図 2** に示す．対象サービスにロ

© 2026 Information Processing Society of Japan

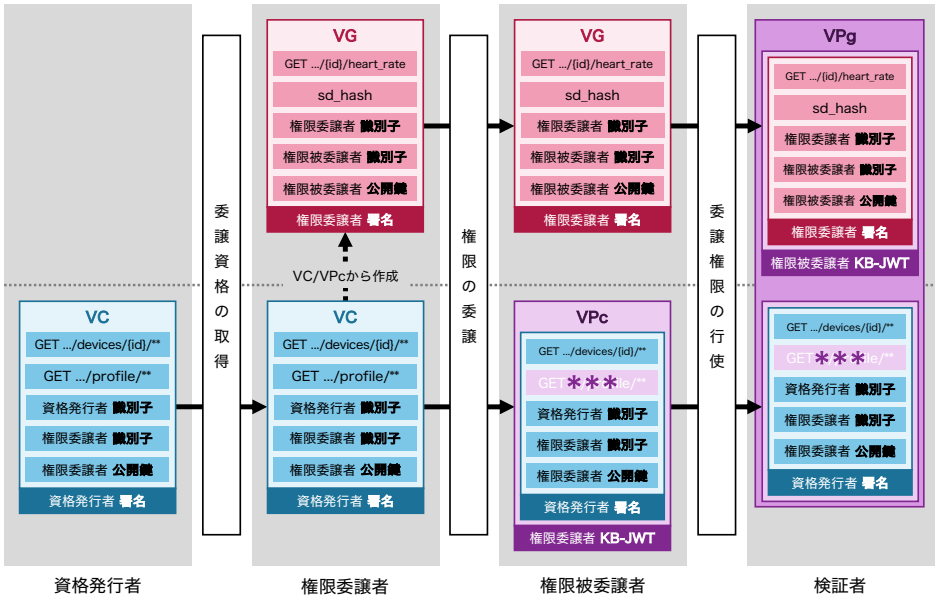


図 1　証明の発行と提示の流れ

Fig. 1　Process for Issuing and Presenting Certificates.

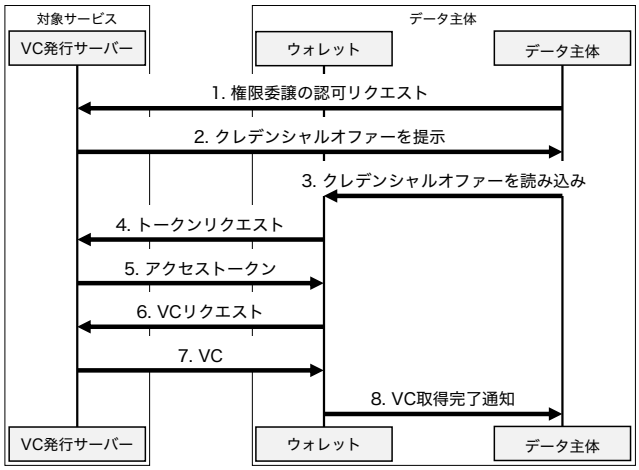


図 2　委譲資格取得フェーズの流れ

Fig. 2　Flow of the Delegation of Authority Approval Phase.

グインしたデータ主体がクレデンシャルオファーを介して，データ主体の公開鍵が紐づいた対象権限に関する VC を受け取る．VC は SD-JWT VC の標準クレームに `authorization_details` を加えた構成である．その中で提案方式で固有の役割を持つのは `sub`, `authorization_details` の 2 つである．`sub` には委譲フェーズで発行される VG の Issuer 識別子が入る．そのため，データ利用サービスがメタデータを解決できるようになっているほか，行使フェーズで VG の Issuer と VC の Holder が同一であることを照合できる．`authorization_details` は表 1 に示す形式の権限を配列形式で複数含んでいる．

3.2.2　権限委譲フェーズ

権限委譲フェーズでは，委譲資格取得フェーズで得た資格を根拠にデータ主体がデータ利用サービスへ権限を委譲する．フェーズの詳細を**図 3** に示す．このフェーズの中では二つの処理を前後半に分けて実行する．

前半は OID4VP の Cross Device フローに基づき，Holder をデータ主体，Verifier をデータ利用サービスとして，データ主体の委譲認可を証明する．提供サービスがデータ主体に返す VPc リクエストには要求する権限情報が含まれており，一回のリクエストで複数の対象サービスから資格を取得した権限について提示を求められる．データ主体ウォレットは VPc リクエストとデータ主体の選択から VPc を作成し，同時に生成される `sd_hash` を保存しておく．また提供サービスは検証した VPc をデータ利用サービスウォレットに保存する．

後半は OID4VCI の Authorization Code フローに基づき，Issuer をデータ主体，Holder をデータ利用サービスウォレットとして，委譲の証明を発行する．データ利用サービスウォレットは VPc 内の VC から取得できるデータ主体ウォレットの Issuer 識別子をもとに，VG の発行をリクエストする．VG の発行にはデータ主体の承認を要するため同期的に返すことができないため，OID4VCI で定義されている Deferred Credential Endpoint を用いて，データ利用サービスウォレットが後から VG を要求できる形を取る．

VG に含まれる内容を**図 4** に示す．VC との差分として，`evidence` と `authorization_details.ev` を追加している．`evidence` は権限保有を示す VC を委譲の根拠として束縛する項目である．検証時の識別に用いる `id` と，図 3 の Step 6 で作成した VPc の `sd_hash` を持つ `digest` が含まれ

© 2026 Information Processing Society of Japan

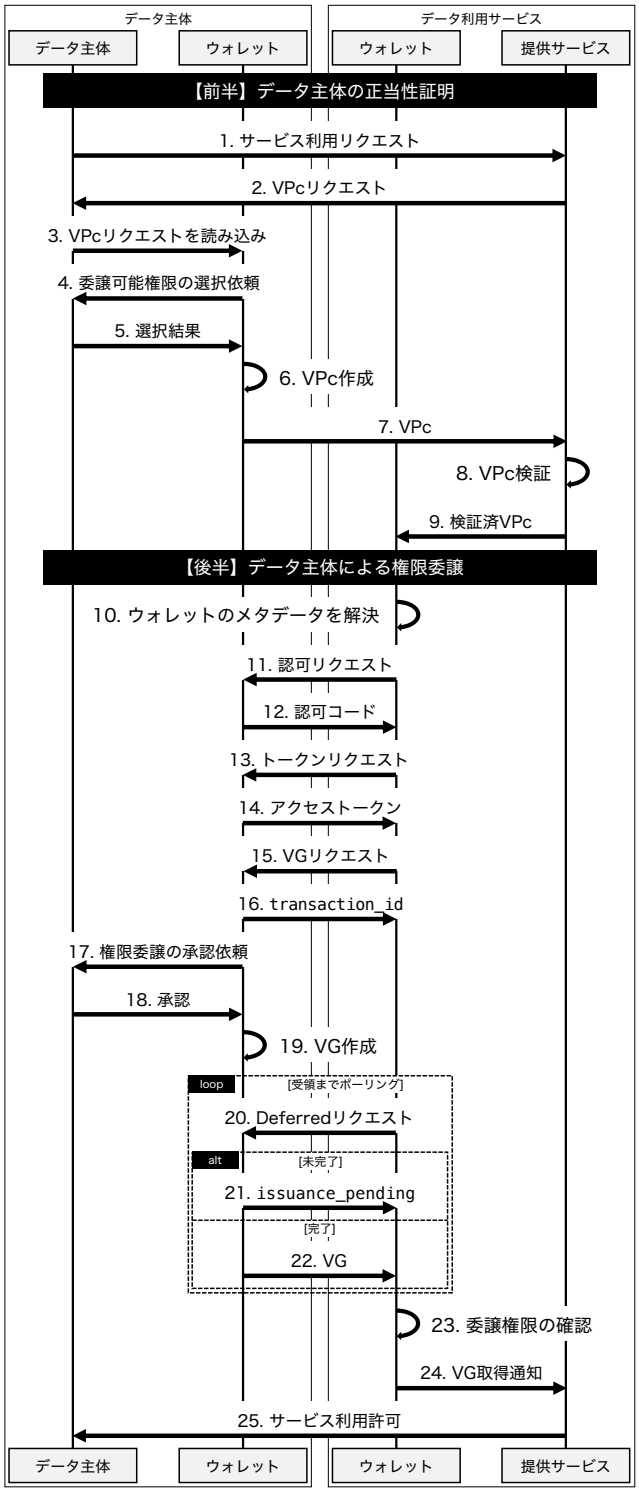


図 3 権限委譲フェーズの流れ
Fig. 3 Flow of the Delegation Phase.

ている．VPc 本体ではなく `sd_hash` を含めることで，VPc の提示から VG 発行までの間にデータ主体が根拠を差し替えることを不可能にする．`authorization_details.ev` は委譲された権限の根拠を識別子で指定する．これにより，行使する権限に必要な根拠だけを指定して提示できる．

VG の有効期限 `exp` には短期の TTL を設定し，既定の有効期間で自然に失効させる．データ利用サービスが権限

```
{
  "authorization_details": [
    {
      "type": "api_access", "actions": ["GET"],
      "locations": ["api.health.example/devices/{id}/heart_rate"],
      "ev": "e1"
    },
    {
      "type": "api_access", "actions": ["GET"],
      "locations": ["api.bank.example/accounts/{id}/**"],
      "ev": "e2"
    },
  ],
  "evidence": [
    {"id": "e1", "digest": "<sd_hash(VPc[A])>"},
    {"id": "e2", "digest": "<sd_hash(VPc[B])>"},
  ]
}
```

図 4 VG に含まれる内容
Fig. 4 Contents of VG.

を継続して行使するには VG の再発行を要するため，データ主体が再発行に応じなければ，TTL の満了とともに委譲は失効する．委譲を終わらせる経路は当事者間で完結し，データ主体側に失効情報を公開するなどの追加のインフラを要さない．

委譲フェーズでデータ利用サービスが行う検証は次の三つからなる．前半では提示された VPc に対して（1）（2）を，後半では取得した VG に対して（3）を行う．以降，対象サービス A が発行した VC を VC_A，VPc のうち VC_A に対応する部分を VPc[A] と表記する．

（1）基本検証 VPc に含まれる各 VC に対して基本検証を行う．

（2）発行者と対象 API の対応確認 開示された権限の `locations` のホストが利用したい API のホストと一致することを確認する．VC の `iss` が当該 API について権限を発行する主体として妥当であることを，データ利用サービスが保持する対応で確認する．最終的な判断は行使時にリソースサーバーが行うが，委譲時点で使えない VC を受け取らないために必要となる．

（3）開示権限との照合 VG に含まれる各権限が，その `ev` が指す根拠において開示された権限に包含されることを確認する（3.1.3 項の（b））．これにより，委譲元が認可された範囲を超えた権限を VG に載せていないことを委譲の時点で確かめられる．

3.2.3 権限行使フェーズ

権限行使フェーズでは，データ利用サービスが VG として委譲された権限を行使し，データを操作する．このフェーズは Holder をデータ利用サービスウォレット，Verifier をリソースサーバーとして OID4VP 上に構成される．フェーズの詳細を**図 5** に示す．提供サービスは行使したい権限に合わせて VPg を提示し．リソースサーバーによる検証の結果得たアクセストークンを用いて API へアクセスする．

図 4 より VG には複数の対象サービスに関する権限が含まれているが，VG の提示では選択的開示により行使先の権限とその根拠のみを提示する．また `sd_hash` は提示全体ではなく VC ごとに計算されるため，VPc[A] だけを取り

© 2026 Information Processing Society of Japan

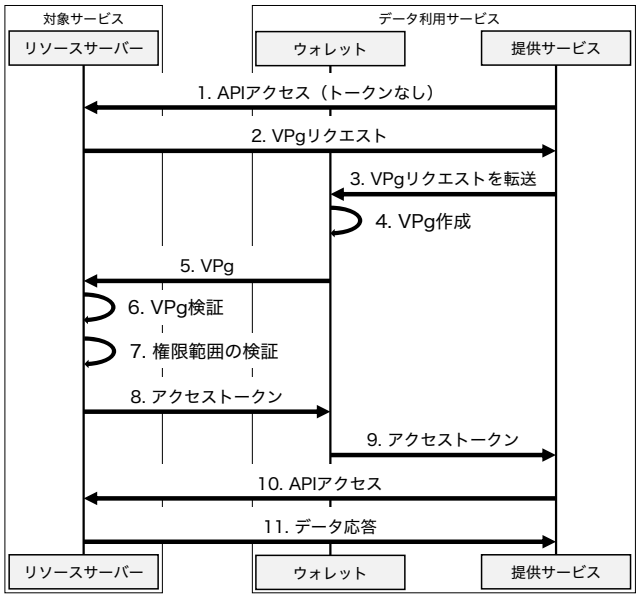


図 5 権限行使フェーズの流れ
Fig. 5 Flow of the Authority Exercise Phase.

出しても根拠として成立することから，VPg には行使先に対応する VPc[A] のみを含める．

リソースサーバーは VPg に対して以下の検証を行う．

（1）提示者の確認 VG の提示に付されたデータ利用サービスの Key Binding 署名を，VG の `cnf` で検証し，提示者が被委譲者本人であることを確認する．

（2）根拠 VC の検証 VPc[A] に含まれる各 VC に対して基本検証を行う．このときリソースサーバーは，自身が VC の発行を認める発行者による署名のみを受理する．ただしデータ主体の KB-JWT は VG の発行者署名が代替するため求めない．

（3）VG の検証 VG の発行者が VC 保有者であることを確認する．まず VPc[A] の VC に含まれるデータ主体の公開鍵を用いて，VG の Issuer 署名を検証する．次に VG の Issuer 識別子と VC の Holder 識別子が同一であるかを確認する．加えて，VG の有効期限が VC の有効期限内であり，かつ検証時点でも有効であることも確認する．

（4）evidence の照合 VPc[A] から `sd_hash` を算出し，行使する権限の `ev` が指す `evidence` の `digest` と一致するかを確認する．これにより，VG による委譲の正当な根拠が存在し，かつ改ざんが行われていないことが示される．

（5）権限範囲の検証 API のアクセスリクエストから順に，行使 ⊑ 委譲 ⊑ 委譲資格 が成り立つことを確認する．表 1 の各段がこれに対応する．

4. 実装と評価

本章では，提案方式が既存の標準仕様の上で実際に構成できることを，概念実証システムの実装を通して確認する．あわせて，追加した検証の処理時間を評価する．安全性については，想定する攻撃を概念実証上で実際に実行し，不正な委譲が拒否されること，および各検証項目を個別に無効化して，いずれも省略できないことを確認する．

4.1 実装詳細

エンティティを，OpenWallet Foundation が開発する TypeScript モジュールの`@openid4vc`[13], および SD-JWT の処理を担う`@sd-jwt` を用いて実装した [14]．対象サービスを 2 つ用意し，それぞれのリソースサーバーが持つ API エンドポイントを実在する API 仕様から抜粋して作成した．一方は英国 Open Banking の Read/Write API 仕様に基づくエンドポイント [15]，他方は HL7 の FHIR R4 の医療情報のエンドポイント [16] である．また，リソースサーバーの検証内に包含判定を実装した．照合は区画単位で行い，リテラルは完全一致，`{param}`は任意の 1 区画に一致し区画を跨がない，`**`は末尾にのみ配置できその位置以下の任意個の区画に一致する，とした．判定前の正規化として，ドットセグメントの解決，パーセントデコード，末尾スラッシュの統一を適用する．なお，権限に列挙されていない HTTP メソッドは拒否する．

以上により，認可・委譲・行使の 3 フェーズが，OID4VCI/OID4VP，SD-JWT VC，RAR という標準仕様の上で完結することを確認した．独自のプロトコルを持ち込まないため，既存の OAuth2.0 ・OpenID Connect を基盤とするシステムの上に構成できる．

4.2 性能評価

3.2 節で示した 3 フェーズの処理時間について，提案方式が追加する行使時検証のコストを評価する．計測は Apple M3（8 コア）・メモリ 24 GB・macOS 26.5.2 のホスト上で行った．各エンティティは Docker（29.6.2）のコンテナとして起動し，コンテナのランタイムは Node.js 24 である．2 つの対象サービスそれぞれの VC 発行サーバーとリソースサーバー，データ利用サービス，データ主体のウォレットの計 6 コンテナが単一ホスト内のブリッジネットワークで通信するため，計測値にはインターネット経由のネットワーク遅延が含まれない．

処理時間の計測結果を**表 2** に示す．委譲フェーズ後半はデータ主体による承認を待つ Deferred 発行を含むため，表 2 の値はポーリング待ちを除いたものである．いずれのフェーズも数十ミリ秒で完了しており，各フェーズが現実的な時間で完了できることが確認できた．

4.3 安全性の評価

提案方式には，委譲時にデータ利用サービスが認可の証明を検証する点と，行使時にリソースサーバーがその根拠まで遡って検証する点の 2 つの検証点がある．本節では，

© 2026 Information Processing Society of Japan

表 2 フェーズごとの処理時間（サンプル数 30）
Table 2 Processing Time by Phase (30 Samples).

フェーズ	処理時間（ms）
資格取得	17.57 ± 9.20
委譲（前半）	18.81 ± 4.44
委譲（後半）	31.58 ± 6.12
行使	11.68 ± 8.18

この 2 点が不正な委譲を実際に拒否できること，および設計した各検証項目がいずれも省略できないことを，想定される攻撃シナリオを概念実装上で実行して評価する．

攻撃者は資格発行者（VC 発行サーバー）・権限委譲者（データ主体）・権限被委譲者（データ利用サービス）・第三者の 4 者を想定し，いずれも他者の秘密鍵を持たないものとする．また，権限委譲者と権限被委譲者が結託する場合も含める．発行者の鍵漏洩，相関耐性の評価，多段委譲は対象外とする．

シナリオと結果を**表 3** に示す．項目番号は 3.2.2 項および 3.2.3 項のものである．全項目を有効にした状態ではシナリオ 0 のみが受理され，各検証項目を 1 つずつ無効化すると，表 3 に示した項目を無効化した場合にのみ対応するシナリオが受理された．8 つのシナリオと 8 つの検証項目は 1 対 1 対応しており，いずれの項目にも代替がなく省略できないことが確認された．シナリオ 1–3 が委譲時に拒否されたことは，委譲時の検証が機能していることを示す．またシナリオ 2・3・6 の権限範囲の逸脱は権限の連鎖でも捕捉され（要件 2），シナリオ 8 は権限委譲者と権限被委譲者が結託して委譲時の検証を経ない場合でも，行使時の根拠 VC の検証によって拒否されることを示す．

シナリオ実行に加え，権限の秘匿について，2 つの対象サービスから VC の発行を受けて 1 つの VG に委譲し，行使時には行使先に対応する権限と根拠のみを開示して，いずれの対象サービスに対してもアクセスが成立することを確認した．他方の対象サービスの認可権限は VPg に含まれないため，ある対象サービスへの行使が他の対象サービスの発行権限を露出しない．

資格発行および委譲の失効について，失効の判定はキャッシュ済の Status List に基づくため，キャッシュが更新されるまでの間は失効した VC が受理される．また本節が示すのは，設計した検証手順が想定した攻撃を拒否し，各項目が必要であることであり，設計そのものの安全性証明ではない．

5. 関連手法との比較

提案方式の比較対象として 3 つの既存手法を取り上げる．OAuth2.0 はリソース所有者の同意のもとで認可サーバーが第三者クライアントへアクセストークンを発行する枠組みであり [5]，拡張仕様の Token Exchange は `act` ／ `may_act` クレームによって委譲の発生と事前認可を表現し，認可サーバーがアクセストークンの交換と発行を行う．User-Managed Access（UMA）は所有者が事前に定めたポリシーに基づき認可サーバーが第三者へ認可を行うモデルで，所有者がオフラインの場合でも認可が成立する [17]．Čučko らが提案した Verifiable Mandate（VM）は，委譲元が委譲先へ認可サーバーを介さず VM を直接発行し，自身の保有権限を示す資格 VC を同梱する委譲モデルであり，検証者は Policy Engine を備えて複数の VDR を参照する [18]．

本提案の各要件について各手法を評価した結果を**表 4** に示す．要件 1 について，OAuth2.0 は対象リソースの構造を扱わず，UMA は保護対象を個別に指定できるが URL の構造は扱わない．VM はスキーマで，先行研究はデータ項目と CRUD 操作の組でそれぞれ抽象的な表現をしており，いずれも権限を URL で表現しない．提案方式は権限を URL パターンと HTTP メソッドの組で表現し，階層構造とパスパラメーター・パスツリーの区別まで対応している．要件 2 について，まず UMA は検証において資格発行までは遡らない．OAuth2.0 は Token Exchange により委譲の連鎖を表現できるが，判断は最上位のクレームと現在の代理者のみで行われる．VM 委譲元の資格 VC を同梱して確認する点で遡及の枠組みを持つが，判定は属性とポリシーによる評価であり，権限範囲の包含ではない．先行研究も遡及検証の枠組みを持つものの，権限が 1 つの具体的な対象を指すため各段は一致判定のみ行う．提案方式は根拠となる VC と VG を紐づけることで，行使 $\sqsubseteq$ 委譲 $\sqsubseteq$ 資格発行の全段を，細粒度のまま検証者が単独で判定できる．要件 3 について，提案方式以外は権限被委譲者に事前登録を課す．OAuth2.0 と UMA では権限被委譲者が認可サーバーへ，VM と先行研究では各主体の識別子と公開鍵が VDR へ登録される．提案方式は権限被委譲者の公開鍵を VG に紐づけることで登録手続き自体を持たず，検証に必要な鍵が提示と検証者の設定から取得できるため，権限被委譲者が特定の基盤に限定されない．

6. おわりに

本研究では，先行研究の VG 委譲モデルを REST API のアクセス権限へ適用し，委譲の成立にサービスの介在を要さない自己主権型アクセス権限委譲方式を提案した．権限を URL パターンと HTTP メソッドの組で表現し，権限範囲の検証を一致判定から包含判定へ一般化することで，行使 $\sqsubseteq$ 委譲 $\sqsubseteq$ 資格発行 の連鎖を，委譲根拠に束縛したままリソースサーバーが単独で判定できるようにした．また，委譲の証明をデータ主体自身が発行し，各フェーズを OID4VCI/OID4VP の上に構成することで，分散台帳を要さない構成とした．概念実証では実在する API 仕様に基づく 2 つの対象サービスを対象に，資格発行・委譲・行使の 3 フェーズが標準仕様のみで完結することを確かめ，全

© 2026 Information Processing Society of Japan

表 3 攻撃シナリオと拒否された検証項目
Table 3 Attack Scenarios and Rejected Test Items.

	シナリオ	攻撃者	詳細	委譲時	行使時
0	正当な委譲	なし	正規の VC と VG を提示	通過	受理
1	偽の認可	権限委譲者	発行者署名のない VC を根拠に提示	(1)	—
2	対象範囲外の発行	資格発行者	自身の対象サービスが提供しない URL 空間の VC を発行	(2)	—
3	保有しない権限の委譲	権限委譲者	認可範囲を超えた権限を VG に記載	(3)	—
4	根拠の差し替え	権限被委譲者	検証済みとは別の VC を行使時に添付	—	(4)
5	権限委譲者でない者による発行	権限被委譲者・第三者	自身の鍵で VG を発行	—	(3)
6	委譲範囲外の行使	権限被委譲者	委譲外の API アクセスを要求	—	(5)
7	VG の窃取	第三者	他者への提示を再利用	—	(1)
8	委譲時検証の迂回	権限委譲者＋権限被委譲者	委譲時検証を経ずに不正な根拠の VG を使用	—	(2)

表 4 既存手法との比較結果
Table 4 Comparison with Existing Methods.

手法	要件 1	要件 2	要件 3
OAuth2.0[5]	△	×	×
UMA[17]	△	×	×
VM[18]	×	△	×
先行研究 [6]	×	△	×
提案方式	○	○	○

○：満たす／ △：部分的・条件付き／ ×：満たさない

フェーズの処理が数十ミリ秒で完了することを示した．また不正な委譲を想定した 8 つの攻撃シナリオを実行し，いずれも委譲時または行使時の検証点で拒否されること，および設計した検証項目と攻撃シナリオが 1 対 1 に対応し，各項目が不可欠であることを確認した．

謝辞 本研究は JSPS 科研費 JP25K21199, JP23K03847 による研究成果を含む．

参考文献

[1] G20: G20 Osaka Leaders' Declaration, Ministry of Foreign Affairs of Japan (online), available from ⟨https://www.mofa.go.jp/policy/economy/g20_summit/osaka19/en/documents/final_g20_osaka_leaders_declaration.html⟩ (accessed 2026-08-05).

[2] 一般社団法人データ社会推進協議会：DATA-EX 分野間データ連携基盤技術仕様書　第 1 版，（オンライン），入手先 ⟨https://data-society-alliance.org/data-ex/techspecs-v1/⟩（参照 2026-08-05）．

[3] European Parliament and Council: Regulation (EU) 2016/679 (General Data Protection Regulation), Official Journal of the European Union, L 119 (2016).

[4] Ferdous, M. S., Chowdhury, F. and Alassafi, M. O.: In Search of Self-Sovereign Identity Leveraging Blockchain Technology, *IEEE Access*, Vol. 7, pp. 103059–103079 (online), DOI: 10.1109/ACCESS.2019.2931173 (2019).

[5] Hardt, D.: The OAuth 2.0 Authorization Framework, RFC 6749, Internet Engineering Task Force (2012).

[6] 梅田匠，掛井将平，白石善明，齋藤彰一：Verifiable Credentials を介してサービス提供者とデータ主体が 協働するアクセス権限委譲手法の設計，研究報告コンピュータセキュリティ（CSEC） 66，情報処理学会 (2025).

[7] Fett, D., Yasuda, K. and Campbell, B.: Selective Disclosure for JSON Web Tokens, RFC 9901, Internet Engineering Task Force (2025).

[8] Terbu, O., Fett, D. and Campbell, B.: SD-JWT-based Verifiable Digital Credentials (SD-JWT VC), Internet-Draft draft-ietf-oauth-sd-jwt-vc-18, Internet Engineering Task Force (2026). Work in Progress.

[9] Looker, T., Bastian, P. and Bormann, C.: Token Status List (TSL), Internet-Draft draft-ietf-oauth-status-list-21, Internet Engineering Task Force (2026). Work in Progress.

[10] Lodderstedt, T., Richer, J. and Campbell, B.: OAuth 2.0 Rich Authorization Requests, RFC 9396, Internet Engineering Task Force (2023).

[11] Lodderstedt, T., Yasuda, K., Looker, T. and Bastian, P.: OpenID for Verifiable Credential Issuance 1.0, OpenID Foundation (online), available from ⟨https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html⟩ (accessed 2026-08-05).

[12] Terbu, O., Lodderstedt, T., Yasuda, K., Fett, D. and Heenan, J.: OpenID for Verifiable Presentations 1.0, OpenID Foundation (online), available from ⟨https://openid.net/specs/openid-4-verifiable-presentations-1_0.html⟩ (accessed 2026-08-05).

[13] OpenWallet Foundation Labs: oid4vc-ts: TypeScript Implementation of OpenID for Verifiable Credentials, GitHub (online), available from ⟨https://github.com/openwallet-foundation-labs/oid4vc-ts⟩ (accessed 2026-08-05). Version 0.5.4.

[14] OpenWallet Foundation: sd-jwt-js: SD-JWT Implementation in JavaScript, GitHub (online), available from ⟨https://github.com/openwallet-foundation/sd-jwt-js⟩ (accessed 2026-08-05). Version 0.20.0.

[15] Open Banking Limited: Open Banking Read/Write API Specification, Version 3.1.7, , available from ⟨https://openbankinguk.github.io/read-write-api-site3/v3.1.7/profiles/⟩ (accessed 2026-08-05).

[16] Health Level Seven International: HL7 FHIR Release 4 (v4.0.1), HL7 (online), available from ⟨https://hl7.org/fhir/R4/⟩ (accessed 2026-08-05).

[17] Maler, E., Machulak, M. and Richer, J.: User-Managed Access (UMA) 2.0 Grant for OAuth 2.0 Authorization, Kantara Initiative (online), available from ⟨https://docs.kantarainitiative.org/uma/wg/rec-oauth-uma-grant-2.0.html⟩ (accessed 2026-08-05).

[18] Čučko, c. and Turkanović, M.: A Novel Model for Authority and Access Delegation Utilizing Self-Sovereign Identity and Verifiable Credentials, *IEEE Access*, Vol. 13, pp. 115102–115134 (online), DOI: 10.1109/ACCESS.2025.3582312 (2025).

© 2026 Information Processing Society of Japan